指摘内容	対応	対応ページ	対応度
To-Be像と技術スタックの繋がりが不明	1.1で「めざすべき新分析環境の姿」として方向性・ユースケース・基本方針を冒頭に配置。1.4で5目的→7観点→4要件の導出過程を明示。1.5で課題→目的→観点→要件の一気通貫図を追加	1.1, 1.4, 1.5	**対応済み**
As-Is課題との対応関係が不明	1.5に課題→目的→7観点→4要件のフロー図を新設。課題群がどの要件に集約されるか視覚的に示した	1.5	**対応済み**
7観点に具体性がない。一般論かわからない	1.3を3列構成（一般的に求められること / 今回RFPで確認すべきこと / 将来を阻害しないために確認すること）に変更。SOMPOケア固有のシステム名・数値を具体的に記載	1.3	**対応済み**
非構造化データの前提が未言語化	1.1の「基本方針」に「テキスト・映像等も構造化されたものから段階的に管理・活用範囲を広げる」を明記。4要件の要件3に非構造化データ設計枠を含めた	1.1, 1.4(要件3)	**対応済み**
Foundryとの差分が不明確	森田さん資料	§4, 別紙A-8	**森田さん資料で対応**
RFI評価の重み付けがない	2.1で4要件ごとに優先度（最重要/重要）を明示し、各評価観点の優先理由を記載	2.1	**対応済み。**
運用モデルの観点が欠落	1.3の⑤運用性に「専門人材に過度に依存せず運用できること」を明記。1.4の要件2「運用継続性・ガバナンスを確保できること」で対応。1.1基本方針にも「データオーナーは各データドメインごとに明確化」を含む	1.1, 1.3, 1.4(要件2)	**対応済み**
RFP範囲が曖昧	2.1で7観点ごとに評価観点+具体的なRFP依頼事項を明示。「RFPで確認させること」列で回答範囲を具体的に定義	2.1	**対応済み**

1. あるべき姿と対応方針

1.1 めざすべき新分析環境の姿

現在のBI活用を安定的に移行・継続しながら、SOMPOケア固有のデータ資産を管理し、将来的に分析対象を段階的に広げられる共通基盤として整備する。

① 新分析環境でめざす姿

1. **コストの最適化**
   事業規模相応の費用対効果の高い環境を構築。コスト透明性を確保し、将来のスケーリングにも合理的に対応
2. **運用安定化・ガバナンス強化**
   パイプライン可視化・標準化。IT企画部門が主体的に保守運用できるよう監査ログ・アクセス制御・データリネージを整備
3. **持続可能性・再現性の確保**
   特定ベンダーに依存しないサステナブルな基盤を構築。ドキュメント整備と標準化を推進
4. **現場主導のデータ活用**
   プログラミング不要のセルフサービスBI環境を整備。施設長・オーナー部門が自らデータに基づく意思決定を行える状態を実現
5. **AI対応・将来拡張性**
   AI/ML活用を実装可能なアーキテクチャを採用。セマンティック層の整備・AIエージェント連携を見据えた段階的な拡張

② 想定する主なユースケース

今回

カスタムメイドケアサポートダッシュボード / 施設モニター / ナースコール分析 / 施設360

▼

将来

- アセスメント情報ヒアリング音声の自動フォーマット化
- ケアプラン作成の自動化・標準化
- カメラ・センサーの最大活用（施設装備標準化）
- 介護リアルデータのカスタムメイドケア活用（重度化予測等）
- 社内問合せAI（RAG）

③ めざす姿を考える上の前提・スコープ

移行スコープ	ータ統合・加工・ガバナンス機能および既存BI資産を業界標準設計で継承する。活用が限定的な機能はBIツールやAI活用で軽量に代替
管理対象	SOMPOケア固有のデータ資産を管理対象とする。テキスト・映像等の非構造化データの生データも保管・カタログ登録の対象とする（Bronze層に格納） NLP/画像解析等の加工処理は難易度が高くユースケースごとに個別設計が必要。具体的な加工処理は段階的に内製化する
データオーナー	データオーナーは各データドメインごとに明確化し、各主体が責任を持って管理できる状態をつくる。中央集約だけでなく横断的に連携・活用できる構造を目指す
分析の範囲	ダッシュボード閲覧（施設長・本部管理者 ）、セルフBI（ミドル部門・経営企画等）、アドホックSQL分析（IT企画部・データ分析チーム）を今回の自走範囲とする。 AI/ML分析は基盤として枠を確保し、段階的に拡張

1.2 AI時代のデータ基盤選定

重要なのは特定のLLMや個別ツールの選定ではなく、データを適切な権限で蓄積・加工・探索できる基盤の選定である。

① 今回のRFPにおける基盤選定の意味

今回のRFPの主目的は、Foundry基盤からの確実な移行である。

一方で、移行先の基盤は、今後のデータ活用・AI活用の自由度を決める。

したがって、今回の基盤選定は単なる移行先比較ではなく、

今後の拡張余地をどこまで確保できるかを見極める判断 でもある。

② AI時代に基盤で重要になること

AI活用の成否はモデル単体の性能だけでは決まらない。実務で継続的に価値を出すには、基盤として次の条件が必要になる。

- 必要なデータが継続的に蓄積されること
- データが加工・検索・横断利用しやすい形で整備されていること
- 施設・部門・職種に応じた権限制御が効いていること
- 監査や運用ルールを含めて継続利用できること
- 新しいAI技術や処理方式を追加しやすいこと

③ 変わるもの / 変わりにくいもの

変化の速い領域と、土台として価値が残る領域を分けて考える。

変わるもの: LLM、AIエージェント、AI関連ツール、実装パターン

変わりにくいもの: データの蓄積・加工・探索、権限制御、ガバナンス

技術では解決しないもの: 運用体制の設計、ナレッジの属人化解消、利用者の教育・定着

④ SOMPOケアにおける示唆

SOMPOケアは、多数施設にまたがる業務データ・介護記録・センサーデータを保有しており、基盤が整えばAI進化の恩恵を大きく受けられる。

一方、基盤選定を誤ると、将来の活用が特定ベンダーや構成に制約され、追加対応や再設計が必要になる。

「今動くか」「Foundryより安くなるか」「社内で回せるか」「将来を阻害しないか」 をあわせて見る必要がある。

1.3 AI活用も見据えたデータ基盤に求められる7つの観点

今回の選定対象はデータ基盤であるが、将来のAI活用やデータ連携の広がりも阻害しないことを前提に、7つの観点で必須要件と加点要件を整理する。

観点	必須要件	加点要件(将来を阻害しないために今回確認すること)
① 連携性	ほのぼの、Axist、眠りSCAN、ナースコール等の既存接続に対応できるか	IoT・映像等の連携先が増えても大きな再設計なく拡張できるか
② 安全性	要配慮個人情報を扱う前提で、施設・部門・役割単位のアクセス制御、監査ログ、権限管理を実現できるか	AI活用時にも同様の統制を維持できる構造か
③ 進化性	Foundry固有制約から脱却し、標準的なデータ形式・疎結合な構成で継続運用できるか	LLMや構造化処理を追加しても全体を大きく作り直さずに拡張できるか
④ 実効性	219本のパイプライン移行、既存BI再現、大容量データ移行を現実的に実現できるか	テキスト・映像等の活用対象が広がっても基盤側で受け止められるか
⑤ 運用性	データの所在・定義・更新状況を把握できるか。セルフサービスBI環境が整備され、施設長・ミドル部門がIT部門への依頼なしにデータ閲覧・分析できるか。IT企画部・データチームがSQLでアドホック分析を完結できるか	AI活用時にも利用データや処理内容を追跡できる構造か
⑥ 開発速度	開発環境の標準構成、権限付与、PoCから本番への移行を進めやすい構成か	テキスト分析や映像構造化などの新ユースケースを段階的に追加しやすいか
⑦ スケール	約1,000施設規模、利用者数増、分析対象拡大に対応できるか	将来、AI活用が増加しても性能・運用・コスト面で破綻しないか

※ なお、AI活用における将来の詳細論点は、後続の「AI活用に向けた将来論点」で別途整理する。

1.4 貴社ニーズに合わせた優先選定要件

1.3で整理した7つの観点をそのまま並列に扱うのではなく、SOMPOケアの現行課題と目的に照らして、今回の選定で特に重視すべき4つの優先要件に束ねて判断する。

優先選定要件	主に対応する観点	今回特に重視する確認事項	この要件を置く理由
要件1 **妥当で説明可能なコスト構造であること**	⑤運用性 ⑦スケール	コスト体系の透明性、データ量増加・利用拡大時のコスト予測可能性、投資対効果の定量的な説明可能性を確保できるか	事業規模に見合う合理的なコスト構造がなければ、継続投資の正当性を説明できないため
要件2 **開発・移行を確実に実現できること**	①連携性 ④実効性	既存ソース接続、219本のパイプライン移行、既存BI再現、大容量データ移行を現実的に実現できるか	移行そのものが成立しなければ、以降の活用に進めないため
要件3 **運用継続性・ガバナンスを確保できること**	②安全性 ⑤運用性	内部チームで日常運用が完結し、 施設長・ミドル部門によるダッシュボード閲覧・セルフBI、 IT企画部・データチームによるアドホック分析が自走できること。 全拠点（1,000+）のRLS・要配慮個人情報保護が実現すること	移行後にベンダー依存が再発すれば、Foundryと同じ課題を繰り返すことになるため 要配慮個人情報を扱う以上、安全性はmust
要件4 **将来のAI活用を阻害しない拡張性があること**	③進化性 ⑥開発速度	標準的なデータ形式、疎結合な構成、PoCから本番へ進めやすい構成、新ユースケースを段階的に追加しやすい構造か	今回は移行が主目的でも、基盤選定が将来のAI活用余地を左右するため

1.5 課題・目的・観点・要件のつながり

4つの優先選定要件は、現行環境の具体的な課題を起点に、達成すべき目的と基盤選定の観点を通じて整理したものである。

課題	目的	7観点	4要件
過剰コスト コスト体系不透明 データ量増加時のコスト増 投資対効果説明困難	①コストの最適化	⑤運用性 ⑦スケール	**要件1** 妥当で説明可能なコスト構造であること
Foudny固有制約 予告なしUI変更 処理ブラックボックス	③持続可能性 再現性の確保	④実効性 ③進化性	**要件2** 開発・移行を確実に実現できること
現場データ活用の停滞 セルフ分析不可、BI分散 データ所在不明 IT企画部門に依頼集中	④現場主導の データ活用 ②運用安定化 ガバナンス強化	⑤運用性 ②安全性	**要件3** 運用継続性・ガバナンスを確保できること
AI活用を前提とした 設計不足 非構造化データ活用余地不足 将来AI拡張性不足	⑤AI対応 将来拡張性	③進化性 ⑥開発速度 ⑦スケール	**要件4** 将来のAI活用を阻害しない拡張性があること

2. RFPの評価観点

2.1 RFP評価観点・依頼事項

優先要件からRFPで特に注視すべき評価観点と確認事項を整理

4要件	優先度	優先理由	評価観点	評価内容
要件1 妥当で説明可能なコスト構造であること	最重要	現行環境の見直し理由にコストが大きく、継続投資の正当性を説明できることが必要なため	開発・運用コスト	試算の妥当性。初期費用、月額・年額費用、従量課金の内訳と前提条件が明確で、費用試算に妥当性があるかを評価する
要件2 開発・移行を確実に実現できること	最重要	移行が成立しなければ、以降の活用に進めないため	推進体制	本PJを推進する体制が、役割分担・人数・主要メンバーの経験・実績の観点で十分かを評価する
			移行実効性	Foundryパイプラインや関連資産について、移行方針が妥当か、想定工数や移行ステップに実現性があるかを評価する
			資産継承性	PySpark資産の移植/書き換え方針の妥当性を評価する。Tableauや既存ダッシュボードの再現性を評価する
			移行安全性	段階移行、並行稼働、切戻し、テスト、移行リハーサル等を含め、安全に切り替えられる計画かを評価する
要件3 運用継続性・ガバナンスを確保できること	最重要	要配慮個人情報を扱い、特別なSEに依存しない継続運用が必要なため	運用体制	運用監視、再実行、障害切り分け、問い合わせ対応、変更管理等を含め、移行後に継続運用できる体制かを評価する
			ガバナンス	データセット、パイプライン、ダッシュボード等を対象に、カタログ化・リネージ追跡・品質管理できるかを評価する
			セキュリティ	既存認証基盤との連携、権限制御粒度、個人情報マスキング、監査ログの取得範囲を踏まえ、データアクセスを適切に統制できるかを評価する
要件4 将来のAI活用を阻害しない拡張性があること	重要	今回は移行が主目的だが、基盤選定が将来のAI活用余地を左右するため	オープン性	採用するデータ形式や周辺仕様が、特定サービスへの過度な依存を生まず、将来の移行・再構築を阻害しにくいかを評価する
			AI拡張性	AI開発環境の構成が、特定製品や独自実装への依存度を抑えつつ、PoCから本番移行、将来拡張まで見据えた構成になっているかを評価する

2.2 (参考) 6社RFI回答の評価まとめ

RFI時点では、Deloitteが技術面で相対的に有力。
一方で比較継続候補にも固有の強み・確認事項があるため、最終判断はRFPで提案の具体化を求めたうえで行う

会社	現時点の評価	総括	主な懸念/確認事項
Deloitte	有力候補	Sケアデータに基づく詳細試算を提示。OSSベースでロックイン回避。AI/ML・非構造化・開発環境すべてに○評価。24/365運用体制。技術面で穴がない唯一の提案。 ただし運用条件などは要確認	運用条件の具体性（マネージド体制の日本側要員数・対応範囲） Dify等特定ツール依存(databricks構成推奨なので大きな問題ではない)
Fujitsu	比較継続	egaku/Foundry運用経験者が在籍する唯一の会社。現行環境を知っている価値は大きい。インフラ構成も具体的。「移せるか」の不確実性が最も低い。 ただしKozuchi/DIPのロックインとAI拡張性の弱さが懸念	Kozuchi/DIPの独自ロックイン インフラ構成は具体的だが、Foundryパイプラインの移行マッピングは未提示
Sky	比較継続	AI/ML構成提案が最も具体的（Mosaic AI+MLflow+Agent Bricks）。マルチLLM実績も最多。コスト透明性が高い。将来のAI活用を見据えた設計力が強い。 ただし、Tableau非対応がリスク	Tableau非対応（QuickSight固定） 推進体制に具体性がない
Hitachi	比較継続	介護業界のSnowflake実績あり。データ分析民主化の推進計画が具体的。業界理解とユーザー育成の視点は他社にない。 ただしSnowflake単一、PySpark移行の具体計画なし、オープン標準への言及ゼロ	Pyspark移行の工数・実現性 オープン標準への言及ゼロ 統合カタログの言及なし
PwC	比較継続	SOMPOグループとの関係性、ガバナンス設計の具体性、コンサル力は強い。「仕組みづくり」の伴走者としては最適。 ただしAWS単体運用で運用負荷高	運用負荷が高い(AWS郡の個別管理) OSSを明示的に拒否。
SF	優先度低	データ基盤としてはオーバースペック（年62M～248M円）。AI機能は最高額プラン限定。SF閉域のロックイン。CRM連携が主目的なら検討余地があるが、全社データ基盤としては不適合	全社データ基盤としてはオーバースペック（年62M～248M円）。AI機能は松プラン限定。SF閉域のロックイン

2.3 (参考) 6社RFI回答のレビュー (AI観点)

評価観点に照らし、AI観点で6社のRFI回答を評価する

要件	評価項目	Deloitte	Fujitsu	PwC	Hitachi	SF	Sky
要件3 AI拡張	非構造化 データ対応	○ Unity Catalog +OpenSearch	△ DI Basic内 具体設計なし	△ Step3先送り	△ Document AI +Cortex Search	△ Data 360 松プラン限定	○ AI Functions+RAG +AI-OCR
要件2 運用	カタログ ・リネージ	○ Unity Catalog 自動リネージ+品質	○ Unity Catalog DI Basic内蔵	○ DataZone リネージ+MDM	△ Snowflake内蔵 専用カタログなし	△ Data 360 松プラン限定	○ Unity Catalog 松プランのみ
要件2 運用	権限管理	○ Unity Catalog RLS+列マスキング	△ DI Basic内 具体設計なし	○ IAM+Cognito RLS対応	○ Snowflake RLS +マスキング	○ Trust Layer RLS+CRLS	○ IAM（梅竹） Unity Catalog RLS（松）
要件3 AI拡張	AI開発ロックイン	△ Delta Lake/MLflow aws構成での Dify依存に注意	× AI層がKozuchi 独自依存度大	○ AWS標準サービス 移行先あり	△ Snowflake内部形式 移行時は書直し	× SF閉域 全面作直しリスク	○ Delta Lake +Databricks（松）
要件3 AI拡張	マルチLLM	○databricks経由 GPT/Claude/Llama等	× Kozuchi/Takane 独自LLM	△ Bedrock前提 マルチLLM可能	○ Cortex経由 GPT/Claude/Llama等	× Einstein独自 SF内で完結	○ databricks経由 GPT/Claude/Llama等
要件3 AI拡張	AI/ML開発環境・ 将来拡張性	○ Agentic AI構成図 MLflow+Registry Dev/Stg/Prod分離 Notebook統合環境	△ DI Basic内蔵 詳細不明 環境分離の具体設計なし	× AI/MLは将来検討扱い 実験環境の提案なし	△ DataRobot MLOps Snowflake内で完結 環境分離方法不明	× 松プランのみ Einstein Studio 独自閉域環境	○ Mosaic AI+MLflow Agent Bricks 梅竹松で段階的環境提供 松でSandbox設計あり

補足: AI開発ロックインの評価基準

× = 独自製品依存（乗り換え＝全面作り直し）… Kozuchi、Einstein等
△ = 特定PF依存（乗り換え可能だが書き直しコスト発生）… Snowflake内部形式等
○ = 業界標準 or OSS（移行先が複数あり現実的に乗り換え可能）… AWS標準、Delta/Iceberg等

2.4 (参考) 6社RFI回答のレビュー (AI以外の観点)

各社のRFI回答をAI以外の観点で評価

評価項目	Deloitte	Fujitsu	PwC	Hitachi	SF	Sky
コスト透明性/コスト	○（Sケア個別の試算詳細）	△(試算はないが、現時点では不透明なため妥当)	△ (概算のみ)	△（Snowflake従量課金）	△	○（AWS明細型）
既存資産の再利用可能性	○（PySpark native）	○（PySpark native）	○（Glue=PySpark可）	△（Snowpark、PySpark互換だが変換要）	× (PySpark非対応)	○（Glue or Databricks）
ダッシュボード再現性 ※Tableauで要件を満たすか要確認	△（Tableau）	△（PowerBI/Tableau）	△（Tableau推奨）	△（Tableau/PowerBI対応）	△（Tableau）	△（QuickSight）
運用負荷	○（マネージド中心、24/365グローバル保守体制）	△（Databricks主体だが、Sケア主体前提）	×（AWS群の個別管理）	△（SaaS構成だが複数ベンダー管理）	△	○（保守サービスの明記）
データ形式ロックイン	○（Iceberg/Delta開放）	○（Databricks主体）	△（AWS形式依存）	△（SF API依存）	×（SF API依存）	△（フルAWS構成）
移行リスク	○（3パターン段階）	○（egaku運用実績）	○（3 Step移行）	△（介護業界Snowflake実績あるが移行計画詳細不足	△（情報不足）	○ (実績多)
セキュリティ	○（Unity Catalog+IAM）	○（IAM+CloudTrail）	○（IAM+Cognito）	○（Horizon + HIPPA準拠Snowflake実績）	○（Einstein + Tableau RLS）	○（IAM+CloudWatch, 松でUnity Catalog)

3. AI活用に向けた将来論点

3.1 AI活用に向けた将来論点

AI活用の進展を見据え、将来追加で検討が必要となる論点を7つの観点で整理する

観点	将来論点	将来の検討事項	いつ検討すべきか
①連携性	HD/グループ会社とのデータ共有や、施設IoT・映像等のリアルタイム連携が必要になる可能性がある	共有方式、リアルタイム連携要否、対象データ、更新頻度、遅延許容、障害時再送	HD/グループ会社側の連携方針が確定した時、またはIoT/映像ユースケースが具体化した時
②安全性	AIがデータアクセス主体となる場合、人と同等のID・権限・監査・ガードレール設計が必要になる	AI主体を含む認可方針、データ分類、監査方針、AI個別権限、入出力制御	AIユースケース検討時
③進化性	AIモデルは短期間で変化するため、精度評価・劣化検知・改善サイクルを前提とした運用が必要になる	評価指標、劣化検知、モデル/プロンプト/RAG構成の管理、切替手順、改善サイクル	本番利用するAI機能・モデルの導入方針が決まった時
④実効性	介護記録テキスト、音声、IoT、映像等の非構造化データは、構造化データと異なる加工設計が必要になる	対象データ選定、原本/加工後データ管理、チャンク化・ベクトル化、品質管理、構造化データとの紐付け	非構造化データを用いる具体ユースケースが決まり、対象データと精度要件が明確になった時
⑤運用性	AIはモデル・データ・プロンプト・ツールの複合体となるため、障害時の原因切り分け、停止判断、コスト管理が難しくなる	ログ/トレース、障害切り分け、停止基準、コスト可視化、監視・運用体制	AIユースケースの実装設計に着手する時
⑥開発速度	AI開発は試行錯誤前提であり、PoC→本番→横展開の標準手順が未整備だと立ち上げが遅延する	標準運用、利用可能データ範囲、権限付与、PoC→本番移行条件	PoC候補具体化時に環境要件を確定
⑦規模	AI/Agent増加で品質・責任・コスト管理が難しくなる	登録管理、標準化、責任分界、コスト管理	複数AI機能が開発された時

※ 体制・人材・制度の論点も別途存在

4. SW構成基本方針/データ・フロー基本方針

4.1 主要コンポーネントの基本構成方針

現行環境で実際に活用されているデータ基盤機能を分解し、業界標準の構成要素に再マッピングした結果、以下の7コンポーネントを採用する。

コンポーネント	構成方針	主に対応する優先要件
インテグレーション	•多様なデータソースを標準的な接続方式で取り込める構成とする •既存連携の再利用を重視しつつ、将来の非構造化データや外部サービス連携の追加にも対応可能とする	要件2 開発・移行 要件4 将来拡張性
ストレージ	•構造化データを中心に、原本・標準化済み・活用用データを統合管理できる構成とする •将来の非構造化データ追加を阻害しないことを前提とする	要件2 開発・移行 要件4 将来拡張性 要件1 コスト構造
オーケストレーション	•ワークフロー実行、スケジュール管理、再実行、監視を一元的に扱える構成とする •個別運用を増やしすぎず運用負荷を抑えやすい構成とする	要件2 開発・移行 要件3 運用継続性・ガバナンス 要件1 コスト構造
変換・処理	•既存パイプライン資産の再利用性・移植性を重視し、処理ロジックを継承しやすい構成とする •将来の追加加工(非構造データ含む)にも拡張可能とし、管理しやすい構成とする。個別実装や属人運用を増やさないことを重視する	要件2 開発・移行 要件4 将来拡張性
ガバナンス	•カタログ、リネージ、権限、監査ログを統合的に管理・運用できる構成とする •データとパイプラインを横断して可視化・統制でき、管理対象、更新責任、監査運用が継続できることを重視する	要件3 運用継続性・ガバナンス 要件1 コスト構造
ML/AI	•ML/AI機能そのものは現時点で存在しないため、実装可能な枠確保を基本とする •将来のML・AIモデル、プロンプト/RAG、推論ログを追加しやすい構成とする •将来的に特定モデルのみや特定技術に限定されない構成とする	要件4 将来拡張性
可視化	•既存BI資産の継続活用は必須としないが、移行後に可視化のクオリティや活用面で負担がない表示が可能な構成とする	要件2 開発・移行 要件3 運用継続性・ガバナンス

4.2 データの段階的な階層設計

Bronze（生データ）→ Silver（クレンジング済）→ Gold（ビジネスロジック適用）の3層構造

設計基準	Bronze層	Silver層	Gold層
目的	**ソースデータの保管 「事実の記録」として不変**	**分析可能な状態に標準化 クレンジング・正規化済**	**ビジネスKPI・BI/ダッシュボード接続用 集計・結合・ロジック適用済**
変換ルール	取り込んだままを保持	型変換・NULL処理・重複排除 カラム名の標準化	集計・結合・ビジネスロジック
品質チェック	到着確認・件数チェック・スキーマ確認	品質ゲート（違反行はアラート）	KPI整合性チェック
テーブル命名例	raw_care_records raw_sensor_data	clean_care_records master_facilities	kpi_facility_monthly dim_staff, fact_care_events

設計原則：各層の独立性を維持し、Bronze → Silver → Goldの一方向依存を徹底

階層に分離することで不具合時の原因特定を明確化。BIツール利用ユーザーにはGold層を提供、分析ユーザーにはSilver層を提供。

4.3 開発・本番環境の分離

安定運用のための環境分離の方針

3階層の環境構成

開発環境 (dev)	ステージング環境 (stg)	本番環境 (prod)
開発者個人の実験・検証	**統合テスト・品質検証**	**本番環境・ダッシュボード接続**
Branch単位で作業	本番同等スキーマで検証	変更はStagingからのリリースのみ
サンプルデータを使用	CI/CDで自動デプロイ	本番データ・施設別アクセス制御

原則：環境間のデータ移動は禁止。コード（パイプライン定義）のみがCI/CDで環境間を移動する

Dev環境では特定時点のスナップショットのサンプルデータを使用して、開発に活用。Stagingで本番同等の環境で検証を行い、合格した場合にProdへリリース。

開発～リリースの流れ

① 開発・実験
開発環境

- Feature Branchで作業
- サンプルデータで開発・検証
- テスト実行

② PR・レビュー
コードレビュー

- PR作成（最低1名承認）
- テスト自動チェック
- mainブランチへMerge

③ ステージング反映・検証
ステージング環境

- CI/CDで自動デプロイ
- 本番同等データで統合テスト
- データ品質チェック通過を確認

④ 本番リリースPR
本番リリースのレビュー

- リリース承認（担当者）
- CI/CDで自動デプロイ

Appendix 目次

別紙A アーキテクチャ・技術選定

A-1 アーキテクチャ：DWH+DL vs Lakehouse
A-2 テーブルフォーマットとは
A-3 UniFormによるフォーマットの収束
A-4 プラットフォーム比較：Databricks vs Snowflake vs AWS
A-5 本プロジェクトにおける各PF評価
A-6 BI層にSnowflakeを挟む必要性
A-7 オーケストレーション：構成パターン比較
A-8 データ変換：Foundry PySpark移植戦略
A-9 セマンティックレイヤー：Ontology代替

別紙B ガバナンス移行マッピング

B-1 データガバナンス：Foundry機能の代替マッピング

別紙C 推奨技術スタック総括

C-1 推奨技術スタック：総括
C-2 技術選定サマリー

1 アーキテクチャ：「DWH + Data Lake」 vs 「Lakehouse」

従来の2層構成を1つに統合した Lakehouse が新しい標準

従来型：DWH + Data Lake（2層構成）

Data Lake
S3 / GCS
生データを安価に保管
SQLで直接分析は不可

→ ETL →

Data Warehouse
Snowflake / Redshift
加工済みデータをBI向けに提供
コスト高・ML/AI非対応

✗ データが2箇所に存在 → 同期コスト・整合性の問題

✗ DWHライセンス + Lake運用 = 二重課金

✗ ML/AIはDWHから取り出して別環境で実行が必要

Lakehouse（統合型 — 推奨）

Lakehouse
Databricks + S3 / Palantir Foundry（現行）

- **保管** — Data Lakeの安価なストレージ
- **分析** — DWHの高速なSQL検索
- **加工** — ETLパイプラインを統合実行
- **AI** — ML/AIを同一基盤で
- **管理** — 権限・監査 リネージ統合

✓ データは1箇所 → 管理がシンプル・整合性の問題なし

✓ ストレージはS3価格 + コンピュートは従量課金

✓ BI・ML/AI・ETLが同一基盤で完結

観点	DWH + Data Lake（2層）	Lakehouse（統合）
データの場所	2箇所（Lakeに保管 → DWHにコピー）	1箇所（S3上にすべて統合）
コスト構造	DWHライセンス + Lake運用 = 二重課金	ストレージはS3価格 + コンピュート従量課金
BI対応	◎ DWH側は非常に高速	○ 近年大幅改善。実用上十分
ML/AI対応	△ DWHから取り出して別環境で実行	◎ 同一基盤でデータに直接アクセス
ガバナンス	○ DWHとLakeで別々に管理	◎ 統合管理（権限・監査・リネージ）
ベンダーロックイン	△ DWH側にロックインされやすい	◎ オープンフォーマット（Iceberg等）で回避可能
市場トレンド	レガシー構成	新規構築の主流

2 テーブルフォーマットとは

S3上のファイルを「テーブル」として扱うための仕組み

テーブルフォーマットなし（S3のみ）

S3バケット

- data/year=2024/month=01/part-001.parquet
- data/year=2024/month=02/part-002.parquet
- data/year=2024/month=03/part-003.parquet
- ...

→ 単なるファイル置き場

→ SQLでの検索・更新・削除が不可能

テーブルフォーマットあり（Iceberg/Delta）

S3 + メタデータ層

✓ スキーマ管理 — 列の追加・変更を安全に実行

✓ ACIDトランザクション — 途中で壊れない書き込み保証

✓ Time Travel — 過去のデータに遡れる（監査対応）

✓ パーティション管理 — 高速検索のための自動分割

✓ 行レベルの更新・削除 — RDBのようなUPDATE/DELETE

主要フォーマット3種の位置づけ

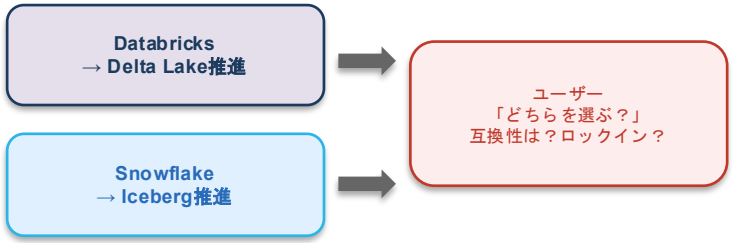
	Apache Iceberg	Delta Lake	Apache Hudi
起源	Netflix → Apache財団	Databricks → Linux Foundation	Uber → Apache財団
位置づけ	業界標準の互換フォーマット	Spark環境での最高性能	ストリーミング特化（ニッチ）
採用状況	975社以上（2026年） Snowflake/Athena/BigQuery対応	Databricksエコシステム中心 Spark上での性能は最速	採用企業は限定的 リアルタイムUpsertに強い
相互運用	マルチエンジン対応が最も広い ベンダー中立性 ◎	UniFormでIceberg互換を実現 Databricks利用時のデフォルト	Iceberg互換を自ら追加 本PJでは選択肢外

2 補足：UniFormによる「フォーマット」の収束

別紙A-3

Delta Lakeで書き込み → 自動でIcebergメタデータ生成 → 外部エンジンからIcebergとして読取り可能

以前の状況：二者択一



現在：UniForm（2024年6月GA）で解決

UniFormの制約事項（要認識）

制約	内容	影響度
読み取り専用	外部エンジンからの書き込み不可。書き込みはDatabricks/Delta経由のみ	中
メタデータの遅延	Icebergメタデータは非同期生成。書き込み直後は一瞬のタイムラグあり	低
性能ペナルティ	Snowflakeからの読み取りはネイティブ比で約20%の性能低下	中
Databricks固有機能	UniFormはオープンな業界標準ではなく、Databricks固有の機能	要認識

結論: Databricks利用時：Delta Lake + UniForm（Iceberg互換）を推奨。

Databricksを書き込みプラットフォームとする前提で二者択一が不要に。マルチプラットフォーム時：Icebergを標準化。ベンダー中立性が最も高い。

3 プラットフォーム比較：Databricks vs Snowflake vs AWS Native

別紙A-4

PySpark互換性・AI拡張性・ガバナンスの観点で評価

Databricks
データエンジニアリング + AI の統合環境

Snowflake
SQLの王者。シンプルで速い

AWS Native
部品を自分で組み立てる最安構成

評価軸	Databricks	Snowflake	AWS Native
PySpark互換性	◎ そのまま動く	× Snowparkへ書き換え	○ EMR/Glueで動く
AI/ML	◎ MLflow, Vector Search等 統合	△ Cortex AI 限定的	○ SageMaker+Bedrock
SQL分析性能	○ 十分高速	◎ 最速クラス	○ Athena/Redshift
ガバナンス	◎ Unity Catalog統合管理	◎ Horizon（SF内は強い）	△ Lake Formation
オープン性	◎ UC(コアOSS), Delta+Iceberg	△ 独自形式+Apache Polaris	○ OSSベース
運用負荷	○ サーバーレス化進行中	◎ フルマネージド	× 自前構築・運用
コスト構造	中〜高（DBU+インフラ二重）	中（クレジット制）	低〜中（従量+人件費）
必要人材	2名〜（データエンジニア）	1-2名（SQL人材）	3-4名（インフラ+データ）

3 本プロジェクトにおける各プラットフォームの評価

Foundry移行の要件に照らした適合度

別紙A-5

Databricks
◎ 最も適合度が高い

Snowflake
△ 本プロジェクトでは不利

AWS Native
○ 安いが運用が重い

評価軸	Databricks	Snowflake	AWS Native
PySpark互換性	◎ そのまま動作 （デコレーター除去のみ）	× Snowparkへ全面書き換え RDD/UDF/ML全て移植が必要	○ EMR上で動作可能 ただしGlue PySparkは制約あり
AI/ML拡張性	◎ MLflow, Vector Search Model Serving, Feature Store	△ Cortex MLは発展途上 Databricksに比べ機能限定的	○ SageMaker連携可能 ただし統合度は低い
ベンダーロックイン	○ Unity Catalog OSS化 で懸念が大幅に軽減	× 独自形式でデータ保持 エクスポートにコスト発生	◎ OSSベース S3 + Icebergで中立性高い
運用負荷	◎ 統合プラットフォーム Foundryに最も近い体験	◎ フルマネージド 運用負荷は最小	× 6-8個のサービス統合 障害切り分けが複雑
人材確保	○ Spark/Python人材 市場に比較的多い	○ SQL中心で敷居は低い ただしSnowpark人材は少数	△ AWSインフラスキル 全サービス横断の人材が困難
コスト	○ 中程度 Commit割引で最適化可能	△ 高め PySpark書き換え工数も加算	○ 最安 ただし人件費含むと差は縮小

結論：本プロジェクトではDatabricksを推奨

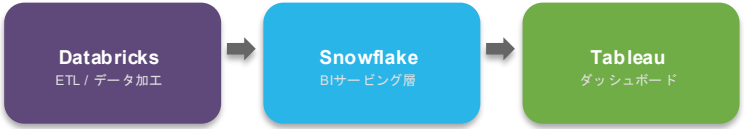
PySpark互換性、AI拡張性、Unity Catalogのオープン性が、Foundry移行の要件に最も合致する。
SnowflakeはPySpark書き換えコスト大、AWS NativeはOSS運用工数が見えにくいリスクあり。

3 BI層にSnowflakeを挟む必要性

よくあるパターン（Databricks ETL + Snowflake）は必要か？

以前はTableau同時多数接続にSnowflakeを挟むメリットがあったが、現在は性能差は実運用では少なく、コストと手間だけ重くなる設計。

パターンA：Snowflakeを挟む構成



⚠ Snowflake追加コスト：年間 1,500-3,000万円（ライセンス + 同期パイプライン構築・運用）

パターンB：直接接続（推奨）

✓ シンプル。追加コストなし。年間 1,500-3,000万円を削減

ポイント：「登録ユーザー数」と「同時接続数」は全く異なる

指標	推定値	根拠
登録ユーザー	800 - 1,000人	施設長700名 + 本部スタッフ
日次アクティブ率	20 - 30%	施設長は毎日閲覧しない。週次/月次チェックが多い
ピーク同時接続	40 - 80人	朝の業務開始時、月初レポート確認時が最大
平常時同時接続	10 - 30人	—

Databricks SQL Serverlessの同時実行能力

10並行クエリ/クラスター × 自動スケーリング
ピーク80人 → 8クラスターで対応（自動）
上限: 1,000並行クエリ — 全く問題なし
2025年: 実ワークロード5倍高速化を達成

さらにTableau Extractで同時接続問題自体が消える

4ダッシュボードはいずれも日次バッチ更新で十分
Extract: 定期的にデータをTableau Serverにキャッシュ
800人がアクセスしてもDatabricksへの負荷はゼロ
Snowflakeを挟む理由が完全になくなる

4 オーケストレーション：構成パターンの比較

Databricks中心であればLakeflow単体が最もシンプルかつ推奨。外部連携の複雑度に応じて構成を選択

パターン1 推奨

Lakeflow 単体 (databricks)

Lakeflow Jobs
- バッチETL
- ストリーミング
- ML推論

↓

Lakeflow Pipelines
- 宣言的ETL
- 品質チェック

パターン2 外部連携が複雑な場合

Lakeflow + Airflow

Lakeflow Jobs
- Databricks内ETL

↓

Airflow DAG
- 外部API連携
- クロスシステム依存

パターン3 統一管理を重視

Airflow/Dagster 一本化

外部オーケストレーター
- 全ジョブ統合管理

↓

Databricks API呼出
- Spark Job投入のみ

評価軸	パターン1：Lakeflow単体	パターン2：Lakeflow + Airflow	パターン3：外部一本化
運用負荷	◎ 最小（フルマネージド）	△ 2ツール運用が必要	△ 外部ツール運用が増加
Unity Catalog統合	◎ ネイティブ統合	○ Databricks内は統合	△ API経由のみ
外部システム連携	△ 複雑な場合に不便	◎ Airflowで柔軟に対応	◎ 全システム一元管理
セットアップ	◎ 追加構築不要	△ Airflow環境の構築が必要	△ 専用環境の構築が必要
コスト	◎ Databricks内で完結	○ Airflow運用コスト追加	○ 外部ツールコスト追加

結論：Databricks中心であればLakeflow単体を推奨

フルマネージドで運用負荷が低く、Unity Catalogとの統合も深い。外部システム連携が複雑な場合のみAirflowを追加。

5 データ変換：Foundry PySpark パイプラインの移植戦略

別紙A-8

既存のPySparkコードはDatabricksへほぼそのまま移植可能。追加ツールの導入は不要

移植の全体像

現行：Foundry

@transform_df
@configure
Foundry固有デコレーター
PySpark DataFrame API

→ デコレーター除去のみ

移行先：Databricks

PySpark DataFrame API
（そのまま動作）
Lakeflow Pipelines
Databricks Notebooks

Databricksが標準提供する機能（追加ツール不要）

IDE / 開発環境	**Databricks Notebooks + SQL Editor** Foundry Code Repositoriesの代替
ジョブ管理	**Lakeflow Jobs（サーバーレス）** Foundryビルドシステムの代替
宣言的ETL	**Lakeflow Declarative Pipelines** パイプライン定義・自動実行
CI/CD	**Databricks Asset Bundles + GitHub Actions** Code RepositoriesのPR→ビルドの代替
リネージ	**Unity Catalog（PySpark列レベル対応）** Foundry自動リネージの代替
テスト	**Databricks DLT Expectations** データ品質チェックの自動化

Foundry → Databricks：機能対応マッピング

Foundry機能	Databricksでの対応	移植工数	備考
PySpark パイプライン（@transform_df等）	PySpark on Databricks	低 デコレーター除去のみ	コアロジックはそのまま動作
Code Repositories（Git + ビルド一体化）	GitHub + Databricks Asset Bundles + Lakeflow Jobs	中 CI/CDパイプライン構築	RFP提案にこの工数が含まれているか要確認
ブランチベースのデータ開発	スキーマ分離 + 開発/本番 プロモーションフロー	中～高 運用フロー再設計	Foundry最大の差別化機能。完全代替は困難 開発→本番の運用ルール再設計が必要
自動データバージョニング	Iceberg/Delta Time Travel + VACUUMポリシー設計	低 保持期間の設計のみ	「全ビルド無期限保持」は非現実的 スナップショット保持ポリシーの設計が必要
自動データリネージ	Unity Catalog Lineage （PySpark列レベル対応）	なし 自動で動作	.toPandas()やRDD操作のみ非対応

6 セマンティックレイヤー：Ontology代替の要否

別紙A-9

ビジネス用語の定義・計算ロジックを一箇所で管理する層。FoundryのOntologyに相当

セマンティックレイヤーなし

- **Tableau** — 入居率 = 入居者数 / 定員（定義A）
- **AIチャット** — 入居率 = 入居者数 / ベッド数（定義B（微妙に違う））
- **Excelレポート** — 入居率 = ???（定義C（不明））

→ 数値が合わない。信頼を失う

セマンティックレイヤーあり

セマンティックレイヤー
入居率 = 入居者数 / 定員（唯一の定義）

- Tableau
- AIチャット
- Excel

→ すべて同じ数値。Single Source of Truth

必要性の判断基準

ユースケース	セマンティックレイヤー	理由
ダッシュボード4つのみ（現状維持）	不要（後から追加可能）	Tableauの計算フィールドで十分
ダッシュボード増加（10+）	推奨	計算定義の食い違い防止
AIチャット（自然言語分析）	実質必須	セマンティック層によりAIのSQL精度が約3倍向上 （学術研究: 16%→54%）
予測AI + 自動レポート	必須	Feature Storeとの連携も必要

主な選択肢：

- **Databricks Metric Views** UC統合・GA済み
- **dbt MetricFlow** OSS (Apache 2.0)
- **Cube.dev** OLAP高速化

結論：初期構築では不要。AIチャットをスコープに入れる場合は初期から設計が必要。

Databricks利用時はMetric Views（Unity Catalog内蔵、2025年後半GA）が最もシンプルな選択肢。

7 データガバナンス：Foundry機能の代替マッピング

別紙B-1

Foundryで統合的に動作していたガバナンス機能を、Unity Catalogでどう代替するか

Foundry機能	役割	Unity Catalogでの代替	対応度	ギャップ / 注意点
Markings（タグ自動伝播）	データ分類タグの自動伝播	Tags（手動 or API設定）	△	自動伝播なし — CI/CDでタグ付け ルールを自動化する運用設計が必要
RLS（行レベルセキュリティ）	施設ごとのアクセス制御（700施設）	行フィルタ + 列マスキング	◎	対応可能。動的RLSも実装可 Foundryと同等のきめ細かい制御
自動リネージ	データの来歴を自動追跡	SQL: 自動追跡 PySpark: 自動追跡（GA済）	○	PySpark lineageはUnity Catalogで 自動追跡。toPandas/RDDは非対応
監査ログ	誰が何にアクセスしたかの記録	System Tables （Audit Logs）	◎	対応可能。SQLで柔軟に分析可 Foundryより分析の自由度が高い
PII検出（Sensitive Data Scanner）	個人情報の自動検出・マスキング	Presidio（OSS） or AWS Macie	△	別途導入・設定が必要 介護データは要配慮個人情報 — 必須

Unity Catalogの位置づけ

コアカタログ機能はApache 2.0でOSS化。ただし高度なガバナンス（列リネージ、動的ポリシー、PII分類）はDatabricks商用版が必要。

◎ そのまま代替可能

- RLS
- 監査ログ

○ 運用設計で対応

- Markings
- リネージ

△ 別途ツール導入

- PII検出

8 推奨技術スタック：総括

Foundry移行における推奨構成

別紙C-1

BI / 分析	Tableau（既存維持）
セマンティック	初期は不要 → AI導入時 Metric Views等を検討
データ変換	PySpark（Python）+ dbt Core（SQL）
オーケストレーション	Lakeflow Jobs（+ 必要時 Airflow）
ガバナンス	Unity Catalog（データ + ML + AI 統合管理）
テーブルフォーマット	Delta Lake + UniForm（Iceberg互換）
ストレージ	S3 + Lakehouseアーキテクチャ

基盤：Databricks Lakehouse Platform

この構成のメリット

1. **PySpark移植コスト最小**
   Foundryデコレーター除去のみでほぼ動作
2. **ツール数最小**
   Databricks + dbt + Tableau の3つが中核
3. **ロックイン回避**
   UC(OSS) + Delta+UniForm(Iceberg互換)
4. **AI拡張性**
   MLflow, Vector Search, Model Serving統合済み
5. **運用人材**
   2名程度（データエンジニア中心）で運用可能

リスク・注意点

- DBU単価の不透明性 — 見積段階で開示を要求
- Markings自動伝播の欠如 — CI/CD自動化で補完
- ストレージ膨張 — VACUUMポリシーの設計必須